6 주차 보고서

전공: 컴퓨터공학과 학년: 4 학년 학번: 20232019 이름: Maksutova Aibike

Random Number Generation Methods other than LCG and Mersenne Twister

In the lab we used LCG and Mersenne Twister (MT). Both are pseudo random number generators (PRNG): they use a formula and a seed, so the same seed always gives the same numbers. Below I explain four of them: Xorshift, PCG, Blum Blum Shub, and hardware random number generators.

1. Xorshift

Xorshift was made by George Marsaglia in 2003. It keeps one number as the state and changes it only with bit shifts and XOR. For example, a 64-bit version is: x ^= x << 13; x ^= x >> 7; x ^= x << 17; and x is the next random number. The period is 2^64 − 1. **Pros:** it is very fast, the code is only a few lines, and it needs very little memory. **Cons:** because it only uses shifts and XOR, the numbers have a linear structure, so it fails some strong statistical tests. Improved versions (xorshift+, xoshiro256**) fix this and are used in some JavaScript engines and games.

2. PCG (Permuted Congruential Generator)

PCG was made by Melissa O'Neill in 2014. Inside, it is an LCG ($X(n+1) = a \cdot X(n) + c \bmod 2^{64}$ or 2^{128}), but it does not output the state directly. Instead, it mixes the state with a permutation function (xorshift and a random rotation that depends on the top bits). This hides the weak low bits of the LCG. **Pros:** fast, small state (16 bytes for PCG64 state), and very good statistical quality; it passes the TestU01 BigCrush tests. NumPy uses PCG64 as its default generator since version 1.17. **Cons:** it is not cryptographically secure.

3. Blum Blum Shub (BBS)

BBS was proposed in 1986 by Blum, Blum and Shub. Choose two big primes p and q (both $\equiv 3 \bmod 4$) and let $M = p \cdot q$. Then $X(n+1) = X(n)^2 \bmod M$, and we output only the last bit (or a few last bits) of each X(n). **Pros:** it is cryptographically secure: predicting the next bit is as hard as factoring M, which is believed to be very hard. **Cons:** it is very slow, because each step needs multiplication of big numbers and gives only a few bits. So it is used for cryptography, not for simulations.

4. Hardware (True) Random Number Generator, TRNG

A TRNG does not use a formula. It measures a physical process that is really unpredictable, such as thermal noise in a circuit, radioactive decay, or timing jitter. For example, Intel CPUs have the RDRAND / RDSEED instructions, which use thermal noise inside the chip. The raw bits are usually post-processed ("whitened") to remove bias. **Pros:** the numbers are truly random and cannot be predicted. **Cons:** it is slower than a PRNG, needs special hardware, and the results cannot be repeated with a seed, so it is hard to debug a simulation with it. The OS (/dev/urandom) uses it as a seed for a secure PRNG.

Comparison

Method	Type	Speed	Statistical quality	Secure	Reproducible (seed)
Xorshift	PRNG	Very fast	Medium	No	Yes
PCG	PRNG	Fast	Very good	No	Yes
Blum Blum Shub	CSPRNG	Very slow	Very good	Yes	Yes
TRNG	Physical	Slow	Truly random	Yes	No

Conclusion

For Monte Carlo simulation we need fast generators with good distribution and a fixed seed, so PRNGs like PCG or Xorshift (and MT from the lab) are the best choice. For security (passwords, keys) we must use secure generators like BBS or a TRNG, because normal PRNGs can be predicted if someone knows the state.